GitLab Functions

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Status: Experiment

GitLab Functions provides reusable units of CI/CD job logic that replace the `script` in a GitLab CI/CD job.

> GitLab Functions is an experimental feature in active development and is subject to breaking changes. For details, review the changelog.

Why functions

When pipelines grow, `script` blocks become hard to maintain. Logic is duplicated across jobs, scripts are fetched from external sources at runtime, and small changes require updates in many places. GitLab Functions are designed to address these problems.

Advantages of functions include:

- Functions are self-contained and versioned. A function is an OCI image that packages the logic, supporting scripts or binaries, and a specification that describes its inputs and outputs. When a step runs, GitLab fetches the function automatically. You don't need to fetch scripts at the start of a job or manually manage external dependencies. When you reference a function at a specific version tag, you get exactly that version every time.
- Functions are reusable across jobs and projects. After you publish a function to an OCI registry, any job can use it with a single `func` reference, without copying and maintaining script files in each repository.
- Functions make data flow explicit. In a `script` block, values are passed between commands through shell variables, which you can set, overwrite, or read in any order. In a `run` list, each step declares its inputs and outputs, and a step can access only outputs from steps that have already run.
- Functions are independently testable. Because a function defines its inputs and outputs, you can run and test it in isolation, without running the whole pipeline.
- Function execution is reliable across platforms. A dedicated agent manages function execution on the build host rather than interpreting a script sent over the wire. This gives functions proper process control, cross-platform consistency, and the foundation for resumable jobs. These capabilities are difficult or impossible to achieve with shell scripts alone.

To reuse existing shell scripts, use the `script` step to run them directly in a `run` list while you migrate incrementally. You can use functions without converting everything at once.

Understand functions

In a traditional CI/CD job, the `script` keyword contains a list of shell commands. The job owns every step and the logic lives directly in the YAML, which describes exactly how to achieve a result. When pipelines grow, this approach becomes difficult to reuse, test, or share across projects.

With GitLab Functions, you use the `run` keyword to declare a list of steps. Each step references a function that contains the implementation, and the job describes what should happen rather than how. Logic exists in the functions, not in the YAML.

The following is an example traditional `.gitlab-ci.yml` for a JavaScript project:

```yaml
```

```
    - npm test
    - npm run bundle
    - BUNDLE_PATH=$(find dist -name '*.js' | head -1)
    - npm run minify -- --input $BUNDLE_PATH
    - npm run deploy -- --artifact $MINIFIED_PATH --env production
```

The same pipeline written with GitLab Functions:

```yaml
build_and_release:
  run:
    - name: validate
      func: registry.gitlab.com/js/validate:1.0.0
    - name: release
      func: registry.gitlab.com/js/release:1.0.0
      inputs:
        environment: production
```

Each job declares what should happen through steps. The functions themselves contain the implementation.

GitLab Functions glossary

This glossary provides definitions for terms related to GitLab Functions.

Function
: A reusable, self-contained package of CI/CD logic. A function contains platform-specific compiled code, a specification that defines its inputs and outputs, and a definition that describes what the function does. The function can run a command or compose other functions.

Step
: A single invocation of a function in a `run` list. A step includes a name, the function reference, any inputs provided, and any environment variables set for that invocation.

Inputs
: Named values you pass into a function when you invoke it as a step. Inputs are declared in the function specification with a type and optional default value.

Outputs
: Named values a function returns after it runs. Outputs are declared in the function specification and written to the output file during execution.

Environment variables
: Variables available to a function at runtime. Environment variables can come from the operating system process environment, runner, function definition, step invocation, or a previously run function that exported them.

Rename from CI/CD Steps

GitLab Functions was previously called CI/CD Steps. The feature and its syntax have been renamed.

Old	New
CI/CD Steps	GitLab Functions
`step:` (deprecated)	`func:`
`step.yml` (deprecated)	`func.yml`
`${{ step_dir }}` (deprecated)	`${{ func_dir }}`

`${{ job.<variable_name> }}` (deprecated)	`${{ vars.<variable_name> }}`

Components and functions

Components and functions operate at different levels of the pipeline and solve different problems.

CI/CD Components are reusable at the pipeline level. GitLab includes a component before any jobs run and contributes jobs, stages, and configuration to the pipeline. Components describe what jobs exist in a pipeline.

GitLab Functions are reusable at the job level. They run inside a job and replace the `script`.

Components and functions operate at different levels and complement each other well. A component can define a job and use functions internally to implement it. When you include the component, you get a fully configured job without needing to know how it works. As the component author, you use functions to handle the complexity of what the job does.

Expression syntax

Components and functions use different expression syntax because they are evaluated at different times:

- `$[[ ]]` expressions evaluate during pipeline creation, before any jobs run. Use this syntax for CI/CD inputs and component inputs.
- `${{ }}` expressions evaluate during job execution, just before each step runs. Use this syntax for function inputs, environment variables, and values that depend on runtime state.

Both syntaxes can appear in a CI/CD Component YAML configuration file:

```yaml
spec:
  inputs:
    go_version:
      default: "1.22"
---

my-format-job:
  run:
    - name: install_go
      func: ./languages/go/install
      inputs:
        version: $[[ inputs.go_version ]]                    # resolved at pipeline creation
    - name: format
      func: ./languages/go/go-fmt
      inputs:
        go_binary: ${{ steps.install_go.outputs.go_binary }}   # resolved during job execution
```

Function execution model

Functions are self-contained packages that can accept inputs, return outputs, and export environment variables. Functions run in the environment of your CI job, whether the instance is a host machine or a container. You can host functions locally on the file system, in OCI registries, or in Git repositories.

Each step in a `run` list runs in sequence. Steps communicate with each other through inputs, outputs, and exported environment variables rather than through shared shell state.

Outputs from one step are available to subsequent steps through the `${{ steps.<step-name>.outputs.<output-name> }}` expression. Environment variables exported by a step are available to all subsequent steps. Both outputs and environment variables become available only after the step completes.

1. Resolves the function reference and fetches the function package from the file system, OCI repository, or Git repository.
2. Evaluates any expressions in the step's inputs and environment variables.
3. Executes the function and passes the resolved inputs and environment.
4. Reads any outputs the function wrote to the output file and makes them available to subsequent steps.
5. Reads any environment variables the function exported and adds them to the global environment.
6. Moves to the next step, or stops if the step failed.

Function requirements

To use functions, you might have to install a step runner on the runner executor you use. For more information, see install the step runner manually.

Use functions

Configure a GitLab CI/CD job to use functions with the `run` keyword. You cannot use `before_script`, `after_script`, or `script` in a job when you run functions.

Run a function with a step

The `run` keyword accepts a list of steps to run. Steps are run one at a time in the order they are defined in the list. Each step has a `name`, either `func` or `script`, and optionally, `inputs` and `env`.

Name must consist only of alphanumeric characters and underscores, and cannot start with a number.

Invoke a function

A step can invoke a function by providing the function reference with the `func` keyword. Pass inputs to the function with the `inputs` keyword, and override environment values with the `env` keyword. Use expressions in the `func` value and in the keys and values of `inputs` and `env`.

Functions run in the `CI_PROJECT_DIR` directory unless the invoked function overrides the work directory.

For example, running the echo function below prints the message `Hi Sally!` to the job log.

yaml

```yaml
my-job:
  variables:
    FRIEND: "Sally"
  run:
    - name: say_hi
      func: registry.gitlab.com/gitlab-org/ci-cd/runner-tools/gitlab-functions-examples/echo:1
      inputs:
        message: "Hi ${{ vars.FRIEND }}!"
```

Run a script

A step can invoke a script with the `script` keyword. Environment variables passed to scripts using `env` are set in the shell. Script steps use the `bash` shell, falling back to `sh` if bash is not found. Expressions can be used in the `script` value, and the keys and values of `env`. Script steps run in the `CI_PROJECT_DIR` directory.

Use the script step when you need something custom and simple alongside functions. Internally, functions converts the script to a function invocation and passes the script as an input.

```yaml
my-job:
  variables:
    FRIEND: "Sally"
  run:
    - name: say_hi
      script: echo 'Hi ${{ vars.FRIEND }}!'
```

Function reference

Functions are loaded from the file system or an OCI repository. Loading from a Git repository is supported but deprecated.

Load from an OCI repository

History >

To load a function from an OCI repository, supply the registry, repository, and version (tag). This method is the recommended way to distribute and consume functions.

Function OCI images support multiple platforms. The step runner downloads the image that matches the running platform. If no match is found, the step fails.

```yaml
# prints 'Hi from GitLab Functions'
my-job:
  run:
    - name: echo
      func: registry.gitlab.com/gitlab-org/ci-cd/runner-tools/gitlab-functions-examples/echo:1
      inputs:
        message: "Hi from GitLab Functions"
```

You can also specify a subdirectory and filename in the image if the function is not at the root:

```yaml
# prints 'snoitcnuF baltiG morf iH'
my-job:
  run:
    - name: echo
      func: registry.gitlab.com/gitlab-org/ci-cd/runner-tools/gitlab-functions-examples/echo:1
reverse/func.yml
      inputs:
        message: "Hi from GitLab Functions"
```

To authenticate to private OCI repositories, set the `DOCKER_AUTH_CONFIG` environment variable with a value in Docker config file format. For a working example of authentication as a function, see the Docker Auth function.

Load from the file system

To load a function from the file system using a relative path, start the function reference with a `.`. Paths are relative to the calling function's directory. When you call the function directly from the job, the path is relative to `CI_PROJECT_DIR`.

Start the function reference with a `/` to load a function from the file system using an absolute path.

The path becomes the function directory when the step runs. The function definition YAML must exist in this directory. Optionally, provide the function definition YAML filename if it is non-standard.

- Load from relative directory:

```yaml
- name: my_step
  func: ./path/to/my-function
```

- Load from an absolute directory:

```yaml
- name: my_step
  func: /opt/gitlab-functions/my-function
```

- Load using a custom function definition file:

```yaml
- name: my_step
  func: ./funcs/release/dry-run.yml
```

Load from a Git repository (deprecated)

> ⚠ GitLab plans to remove support for loading functions from Git repositories in a future release. Load functions from an OCI repository instead.

To load a function from a Git repository, supply the URL and revision (commit, branch, or tag) of the repository. To authenticate to the repository, add a username and password to the URL.

Functions must exist in the `steps` subdirectory when you provide the Git function reference as text in `func`. Functions must exist in the `dir` directory when you use the long-form Git function reference, `git`.

Git repositories contain source, not compiled code. Where possible, load functions from an OCI repository.

For example:

- Specify the function with a tag:

```yaml
- name: my_step
  func: gitlab.com/funcs/my-git-repo@v1.0.0
```

- Specify the function with a branch:

```yaml
- name: my_step
  func: gitlab.com/funcs/my-git-repo@main
```

- Specify the function with a directory, filename, and Git commit:

```yaml
- name: my_step
  func: gitlab.com/funcs/my-git-repo/-/reverse/my-func.yml@3c63f399ace12061db4b8b9a29f522f41a3d7f25
```

- Authenticate to Git when fetching:

```
func: gitlab-ci-token:${{ vars.CI_JOB_TOKEN }}@gitlab.com/funcs/my-git-repo@v2.0.0
```

To specify a directory or file outside the `steps` folder, use the expanded `func` syntax:

```yaml
my-job:
  run:
    - name: my_step
      func:
        git:
          url: gitlab.com/funcs/my-git-repo
          rev: main
          dir: my-functions/sub-directory  # optional, defaults to the repository root
          file: my-func.yml                # optional, defaults to `func.yml`
```

Expressions

Use expressions when you need a value that isn't known until the job runs, such as an output from a previous step, a job variable, or a computed value.

Expressions use the `${{ }}` syntax and are evaluated before each function runs. For the full expression language reference, including operators, data structures, and built-in functions, see Moa expression language.

Expressions can be used in:

- Input values (`inputs`)
- Environment variable values (`env`)
- The function reference (`func`)
- Script content (`script`)

Available context

Use the following context variables when using GitLab Functions. For the full context reference, see Moa expression language.

Variable	Type	Description
`env.<name>`	String	The environment when the function runs. Includes environment variables set by the OS, the runner, and any environment variables exported by previously run steps. `env` does not contain CI/CD job variables.
`vars.<name>`	String	CI/CD job variables passed from the runner. Unlike `env`, this variable is not affected by step exports.
`inputs.<name>`	Any	The input values passed to the current function.
`steps.<step_name>.outputs.<output_name>`	Any	Output values from a previously completed step in the current `run` list.
`func_dir`	String	Path to the directory containing the function's definition file. Use to reference files bundled with the function.
`work_dir`	String	Path to the working directory for the current execution.

Examples

- Reference an output from a previous step:

```yaml
run:
  - name: generate_rand
    func: registry.gitlab.com/gitlab-org/ci-cd/runner-tools/gitlab-functions-examples/random:1
  - name: echo
    func: registry.gitlab.com/gitlab-org/ci-cd/runner-tools/gitlab-functions-examples/echo:1
    inputs:
      message: "The random value is: ${{ steps.generate_rand.outputs.random_value }}"
```

- Use a job variable with a fallback default:

```yaml
run:
  - name: deploy
    func: ./deploy
    inputs:
      environment: ${{ vars.CI_COMMIT_REF_NAME == "main" && "production" || "staging" }}
```

Environment variables

Environment variables move between steps in two ways: you set them with `env`, or export them through a function. The difference matters because they have different scopes.

CI/CD job variables are not available as environment variables. Access job variables using `${{ vars.<name> }}` instead.

Set environment variables for a step

Use the `env` keyword on a step to set environment variables for that step and any functions it calls internally. Variables set with `env` are available to that step in addition to all variables already in the environment. If a variable already exists, the value set by `env` takes precedence. Variables set this way are not available to subsequent steps in the same `run` list.

```yaml
run:
  - name: build
    func: ./build
    env:
      BUILD_TARGET: release   # available to build and its child steps only
  - name: test
    func: ./test              # BUILD_TARGET is not available here
```

Use expressions in the keys and values of `env`.

Exported environment variables

When a function writes to `${{ export_file }}`, the variables it writes are exported to all subsequent steps in the `run` list. Functions use this method to share state with later steps.

Exported variables are available through `env` in expressions:

```yaml
```

```
- name: build
  func: ./build
  inputs:
    path: ${{ env.INSTALL_PATH }}   # available because setup exported it
```

Precedence

When the same variable is set in multiple places, the following order applies, from highest to lowest:

1. `env` set in the function definition (`func.yml`)
2. `env` set on the step in the `run` list
3. Exported by a previously run step
4. Set by the runner
5. Set by the OS process environment

Create your own function

To create a function, see create a GitLab Function.

For example functions, see GitLab Functions examples.

Troubleshooting

Fetch functions from an HTTPS URL

An error message such as `tls: failed to verify certificate: x509: certificate signed by unknown authority` indicates that the operating system does not recognize or trust the server hosting the function.

A common cause is a Docker image that does not have trusted root certificates installed. Resolve the issue by installing certificates in the container or by baking them into the job `image`.

You can use a `script` step to install dependencies before fetching any functions:

```yaml
ubuntu_job:
  image: ubuntu:24.04
  run:
    - name: install_certs
      script: apt update && apt install --assume-yes --no-install-recommends ca-certificates
    - name: echo_step
      func: registry.gitlab.com/user/my_functions/hello_world:1.0.0
```

Acceptable use of user licenses

Cookie Preferences